Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5399, Control Systems

Assignment

Prepared by: Yasmin Al Shawawrh.

ID_Number: 1220848.

Instructor: Dr. Mahran Quraan.

Section: 1.

BIRZEIT

June – 2025

Contents:

Table of Figures: ii
Question1: 1
Question2: 3
Question3: 5
Question4: 7
Question5: 10

Table of Figures:

Figure 1:Q1 MATLAB Script code. *1*
Figure 2:Q1 MATLAB Scripts result. *1*
Figure 3: Q1 MATLAB Simulink. *2*
Figure 4:Q1 MATLAB Simulink result. *2*
Figure 5: Q2 MATLAB Script code. *3*
Figure 6:Q2 MATLAB scripts result. *4*
Figure 7: Q2 MATLAB scripts curve. *4*
Figure 8: Q3 MATLAB Script code. *5*
Figure 9: Q3 MATLAB scripts result. *5*
Figure 10: Q3 MATLAB scripts curve. *6*
Figure 11; Q4 by hand solution. *7*
Figure 12: Q4 MATLAB Simulink. *8*
Figure 13: Q4 MATLAB Simulink result. *8*
Figure 14: Q4 MATLAB Script code. *9*
Figure 15: Q4 MATLAB scripts curve. *9*
Figure 16: Q5 part 1 code. *10*
Figure 17: Q5 part 1 curve. *10*
Figure 18: Q5 part 2 code. *11*
Figure 19: Q5 part 2 the desired pole and gain. *11*
Figure 20: Q5 part 1 curve. *11*
Figure 21: Q5 part 3 by hand solution. *12*
Figure 22: Q5 part 3 code. *13*
Figure 23: Q5 part 3 the desired pole and gain. *13*
Figure 24: Q5 part 4 code. *14*
Figure 25: Q5 part 4 curve. *14*
Figure 26: Q5 MATLAB Simulink (uncompensated). *15*
Figure 27: Q5 MATLAB Simulink result (uncompensated). *15*
Figure 28: Q5 MATLAB Simulink (uncompensated) with feedback. *15*
Figure 29: Q5 MATLAB Simulink result (uncompensated) with feedback. *15*
Figure 30: Q5 MATLAB Simulink (compensated). *16*
Figure 33: Q5 MATLAB Simulink result (compensated) with feedback. *16*
Figure 32: Q5 MATLAB Simulink (compensated) with feedback. *16*
Figure 31: Q5 MATLAB Simulink result (compensated). *16*

Question1:

Use MATLAB Simulink and MATLAB scripts to obtain the step response of a unity feedback system with the flowing forward path transfer function

$$G(s) = \frac{1}{s^2 + 3s}$$

MATLAB scripts (code):

```
%Yasmin Al Shawawrh
%1220848
%Q1

A = [1]; %numerator
B = [1 3 0]; %denominator
G = tf(A, B);
T = feedback(G, 1);
step(T);
title('Step Response of Unity Feedback System');
xlabel('Time');
ylabel('Amplitude');
grid on;
```

Figure 1:Q1 MATLAB Script code.

MATLAB scripts result:

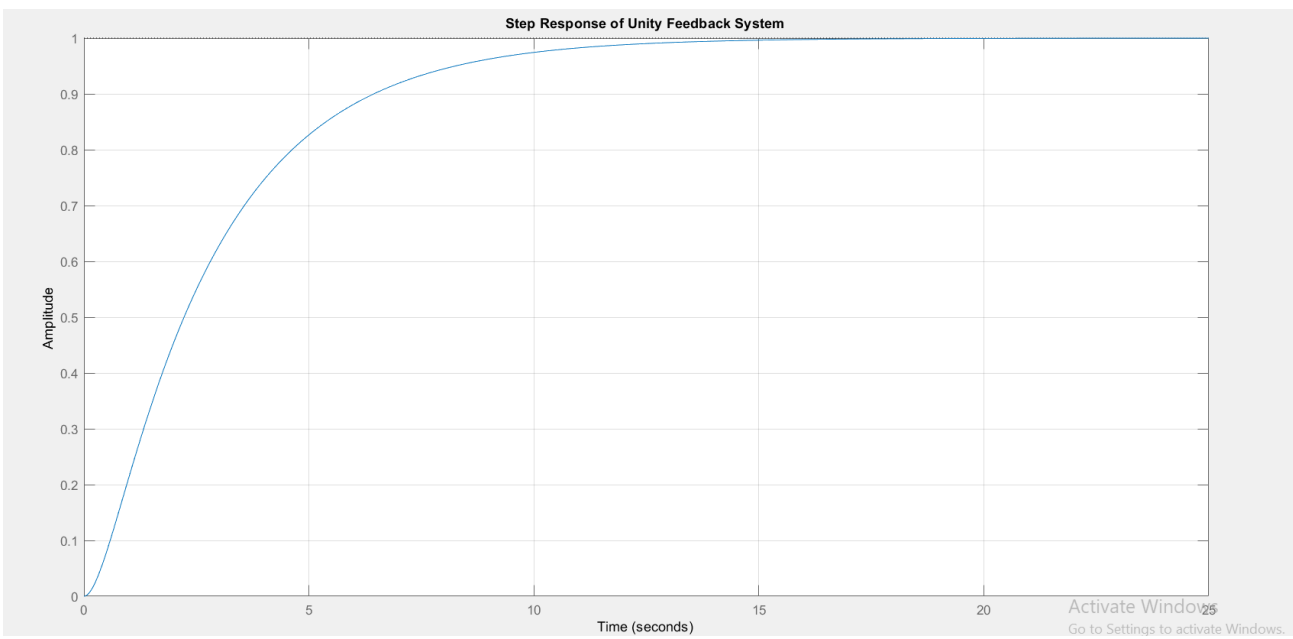

Figure 2:Q1 MATLAB Scripts result.

MATLAB Simulink:

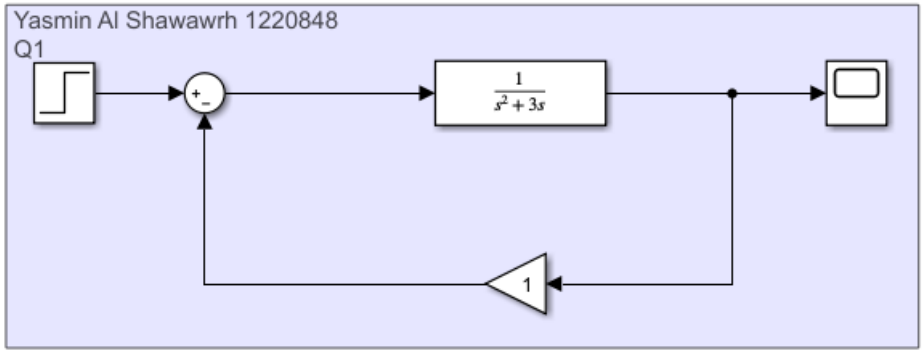

Figure 3: Q1 MATLAB Simulink.

MATLAB Simulink result:

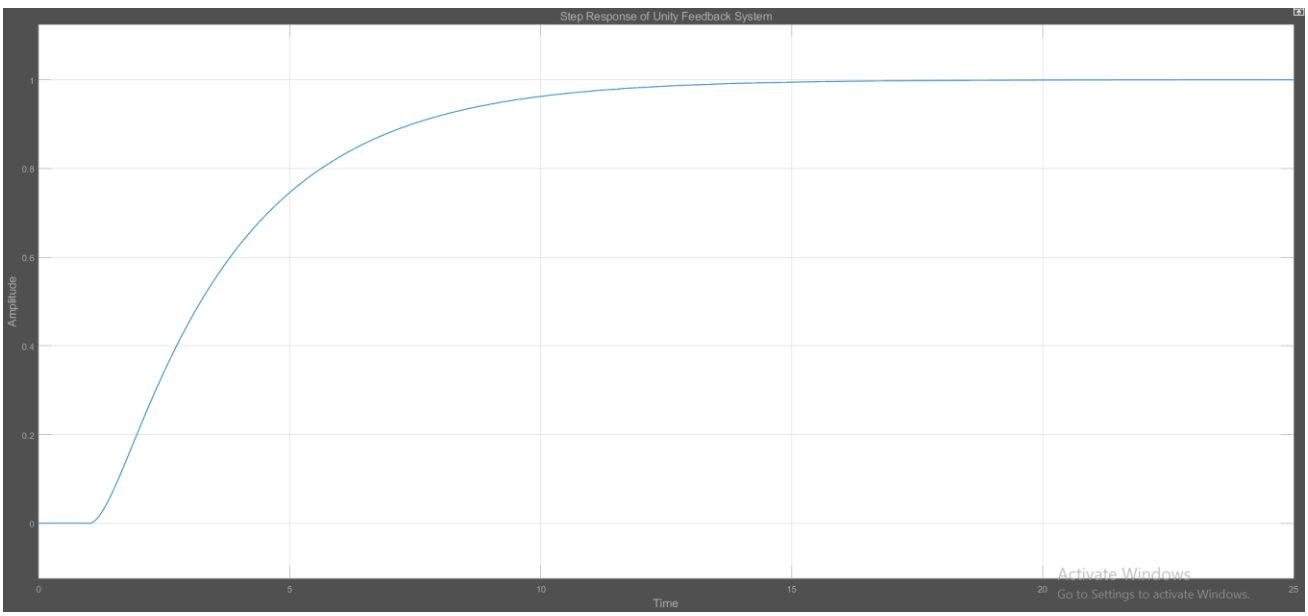

Figure 4:Q1 MATLAB Simulink result.

Question2:

Write the MATLAB scripts to find ζ, ωn, T_r, T_s, T_p, and OS for a unity feedback second-order system and then plot the step response. The forward path transfer function is given by:

$$G(s) = \frac{9}{s(s+2)}$$

$$G(s) = \frac{9}{s^2+2s}$$

MATLAB scripts (code):

```
%Yasmin Al Shawawrh
%1220848
%Q2

A = 9; %numerator
B = [1 2 0]; %denominator
G = tf(A, B);
T = feedback(G, 1); %T(s)= 9/(s^2+2s+9)
wn = sqrt(9);
zeta = 2 / (2*wn);
Tr = (1.76*zeta^3 - 0.417*zeta^2 + 1.039*zeta + 1)/wn;
Ts = 4 / (zeta * wn);
Tp = pi / (wn * sqrt(1 - zeta^2));
OS = exp((-zeta*pi) / sqrt(1 - zeta^2)) * 100;
fprintf('Damping Ratio (zeta): %.2f\n', zeta);
fprintf('Natural Frequency (ωn): %.2f rad/s\n', wn);
fprintf('Rise Time (Tr): %.2f s\n', Tr);
fprintf('Settling Time (Ts): %.2f s\n', Ts);
fprintf('Peak Time (Tp): %.2f s\n', Tp);
fprintf('Overshoot (OS): %.2f %%\n', OS);
figure;
step(T);
stepinfo(T);
title('Step Response of G(s)= 9/(s^2+2s)');
xlabel('Time');
ylabel('Amplitude');
grid on;
```

Figure 5: Q2 MATLAB Script code.

MATLAB scripts result:

```
Damping Ratio (zeta): 0.33
Natural Frequency (wn): 3.00 rad/s
Rise Time (Tr): 0.46 s
Settling Time (Ts): 4.00 s
Peak Time (Tp): 1.11 s
Overshoot (OS): 32.93 %
```

Figure 6:Q2 MATLAB scripts result.

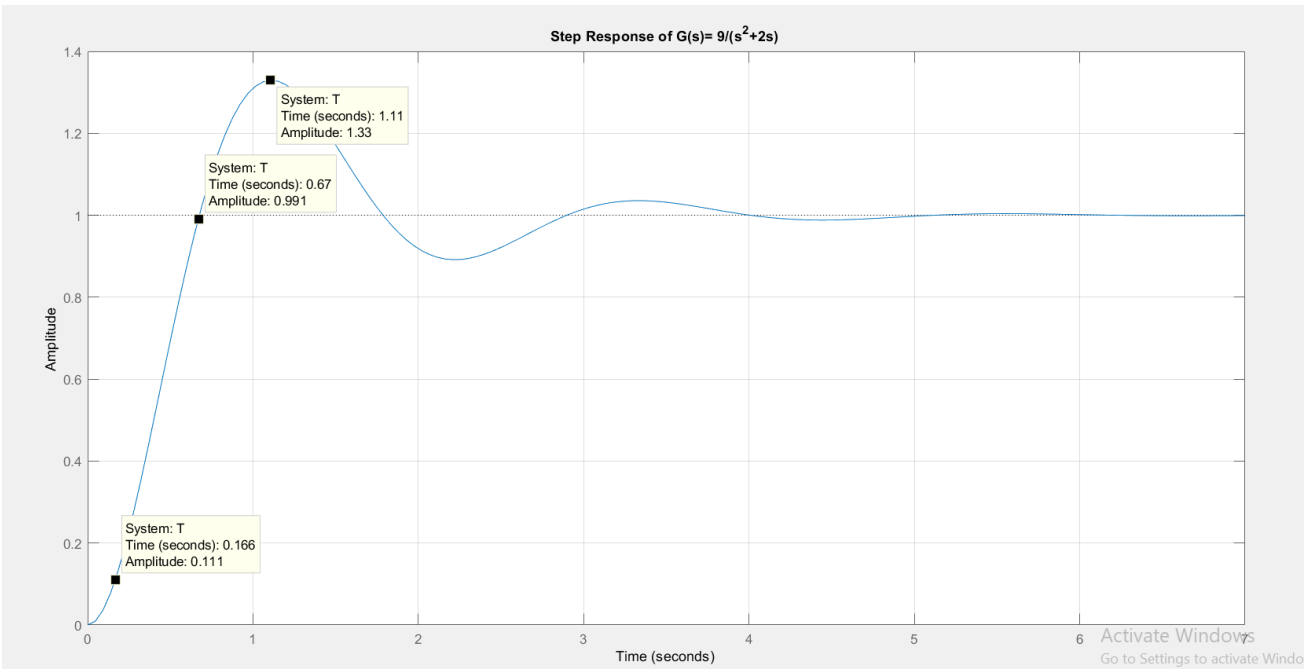

Figure 7: Q2 MATLAB scripts curve.

From the output step response curve, I can find key performance values by looking at the shape of the graph. The **rise time (Tr)** is how long it takes for the curve to first reach the final value. The **peak time (Tp)** is when the curve reaches its highest point. The **settling time (Ts)** is when the curve stays close to the final value without big changes. The **overshoot (OS)** is how much the curve goes above the final value. These values help describe how fast and stable the system responds.

Question3:

Determine the poles for the unity feedback system using MATLAB. The forward-path transfer function of the system is given by:

$$G(s)=\frac{1}{(s+3)(s+7)(s+9)(s+13)}$$

MATLAB scripts (code):

```
%Yasmin Al Shawawrh
%1220848
%Q3

s = tf('s');
G = 1 / ((s+3)*(s+7)*(s+9)*(s+13));
disp('Transfer Function G(s):');
G
poles = pole(G);
zeros = zero(G);
disp('Poles of the system:');
disp(poles);
disp('Zeros of the system:');
disp(zeros);
figure;
pzmap(G);
title('Pole-Zero Map of G(s)');
grid on;
```

Figure 8: Q3 MATLAB Script code.

MATLAB scripts result:

```
>> Q3
Transfer Function G(s):

G =

                    1
  -------------------------------------
  s^4 + 32 s^3 + 358 s^2 + 1632 s + 2457

Continuous-time transfer function.

Poles of the system:
  -13.0000
   -9.0000
   -7.0000
   -3.0000

Zeros of the system:
>>
```

Figure 9: Q3 MATLAB scripts result.

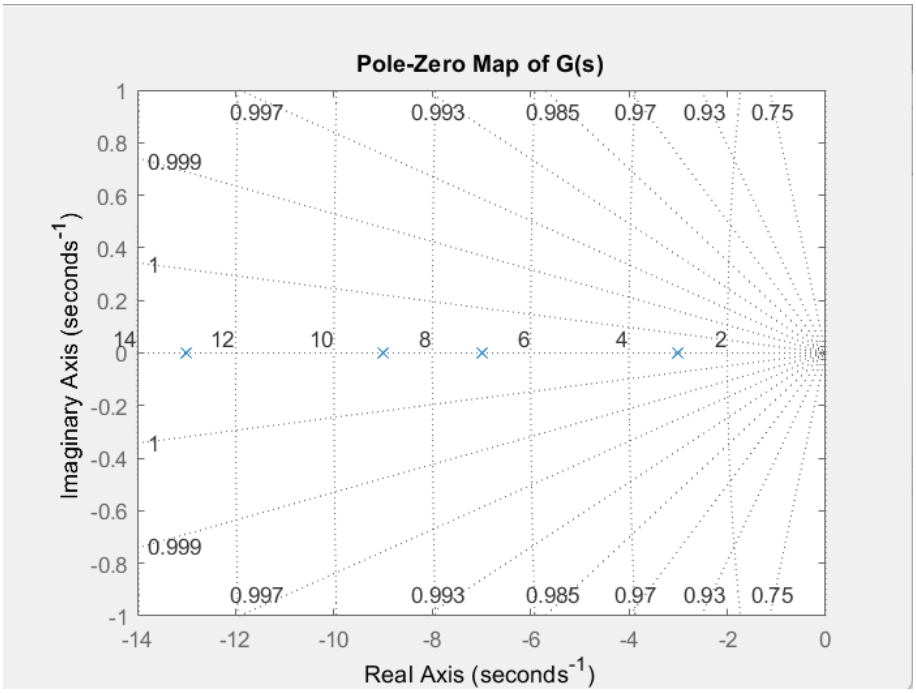

Figure 10: Q3 MATLAB scripts curve.

Question4:

The block diagram of a feedback control system is shown in Figure 1; simulate the system using MATLAB to obtain the step response of the system

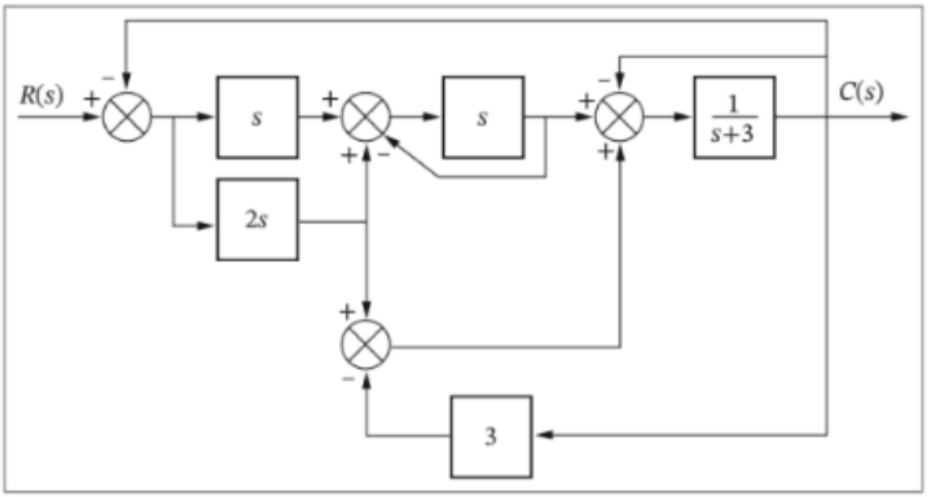

Figure 1

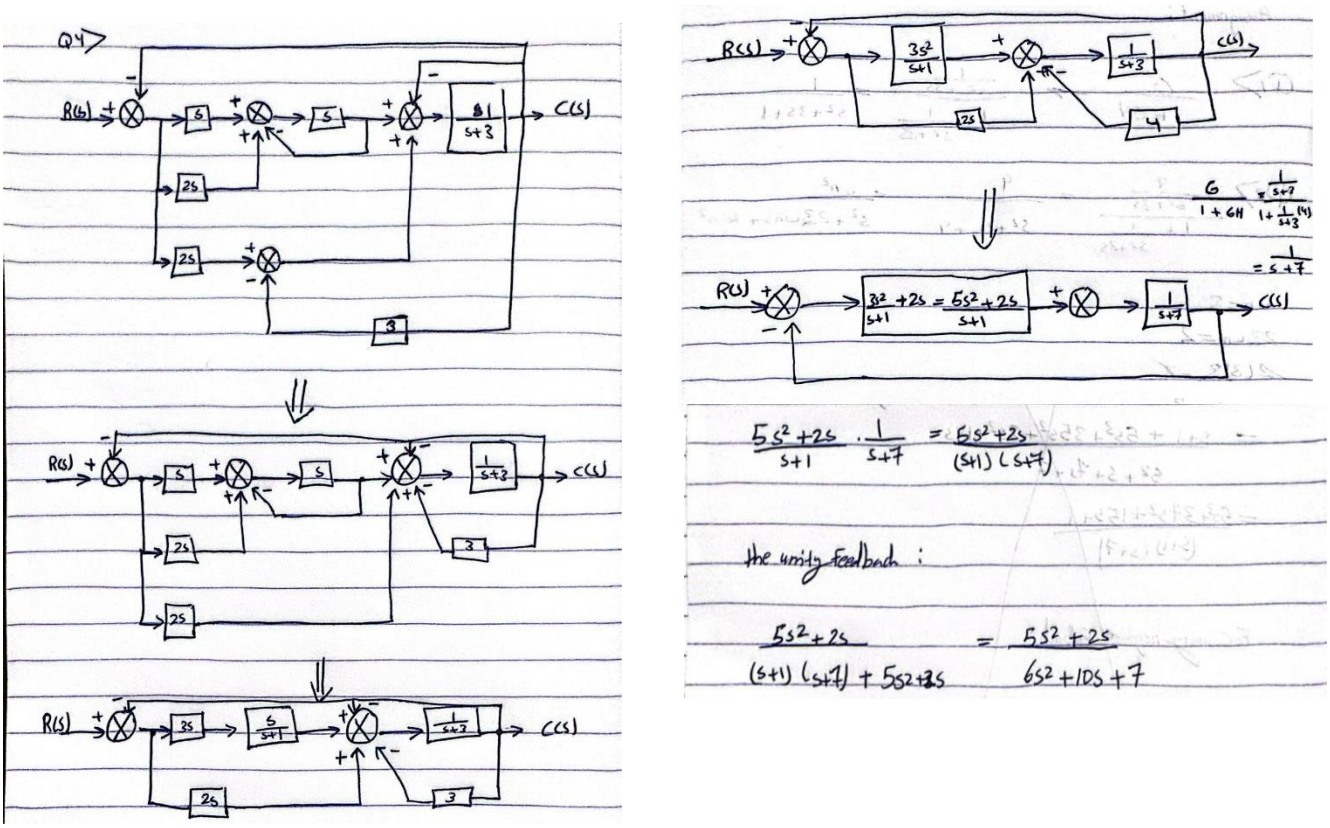

Figure 11; Q4 by hand solution.

MATLAB Simulink:

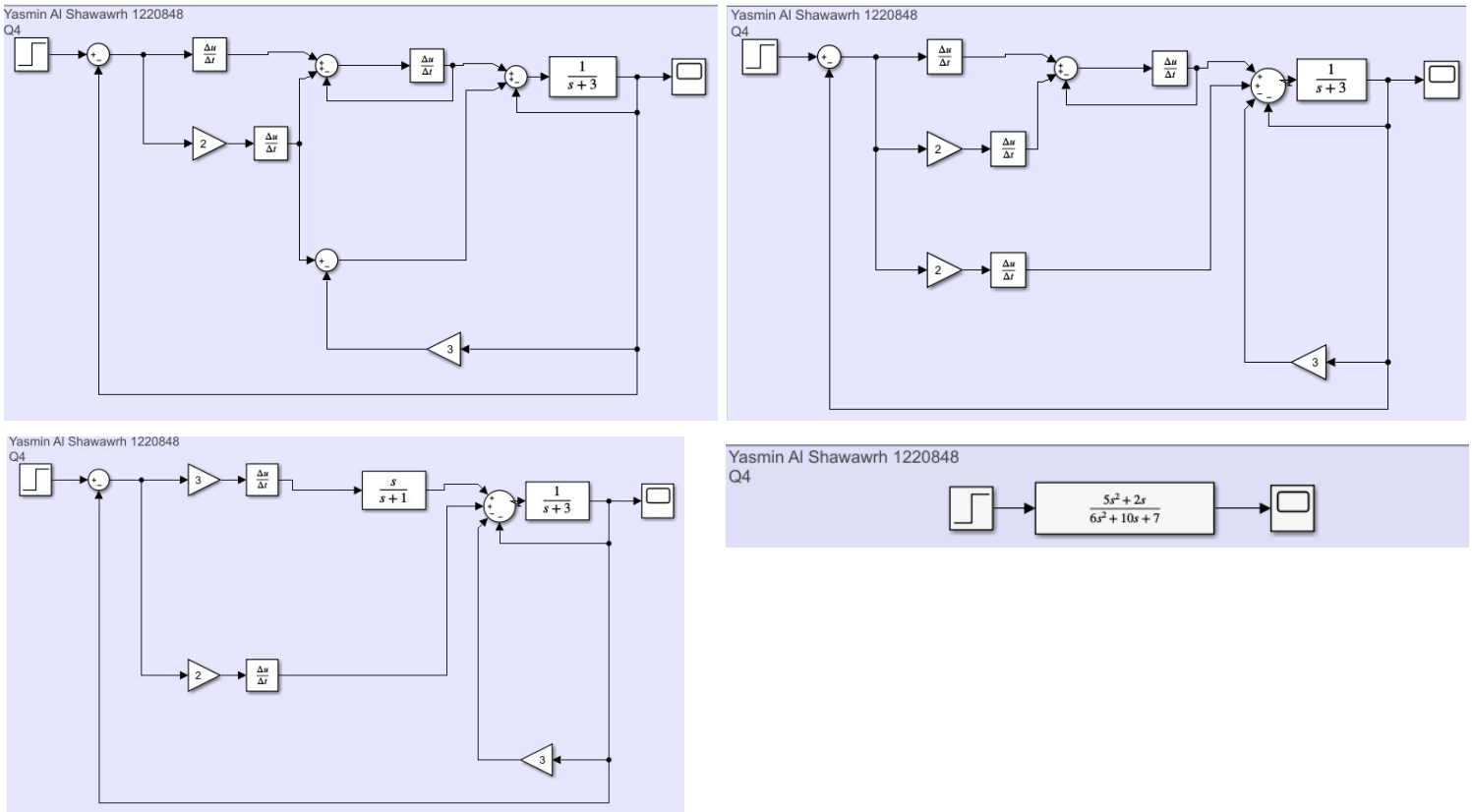

Figure 12: Q4 MATLAB Simulink.

MATLAB Simulink result:

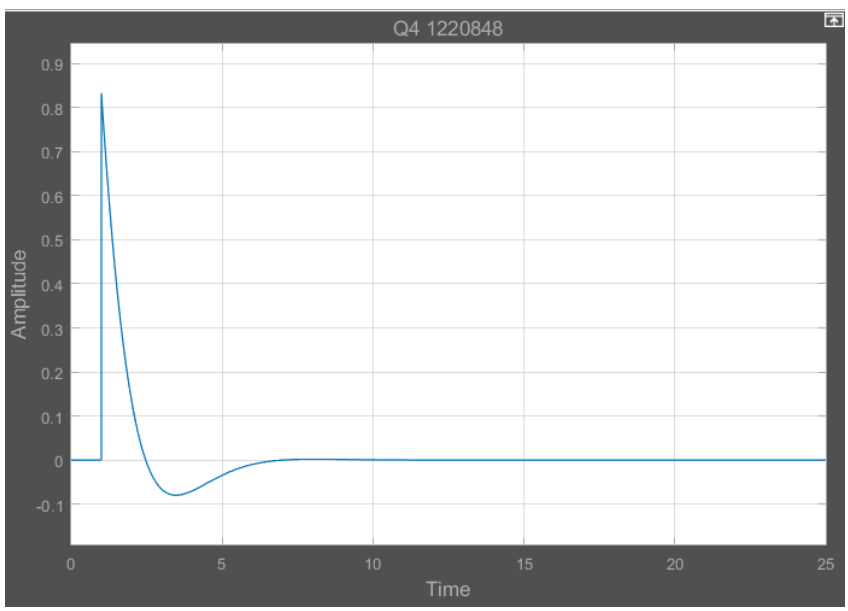

Figure 13: Q4 MATLAB Simulink result.

MATLAB scripts (code):

```
%Yasmin Al Shawawrh
%1220848
%Q4

s = tf('s');
G = (5*s^2 + 2*s) / ((s + 1)*(s + 7));
T = feedback(G, 1);
figure;
step(T);
title('Step Response of Unity Negative Feedback System');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

Figure 14: Q4 MATLAB Script code.

MATLAB scripts result:

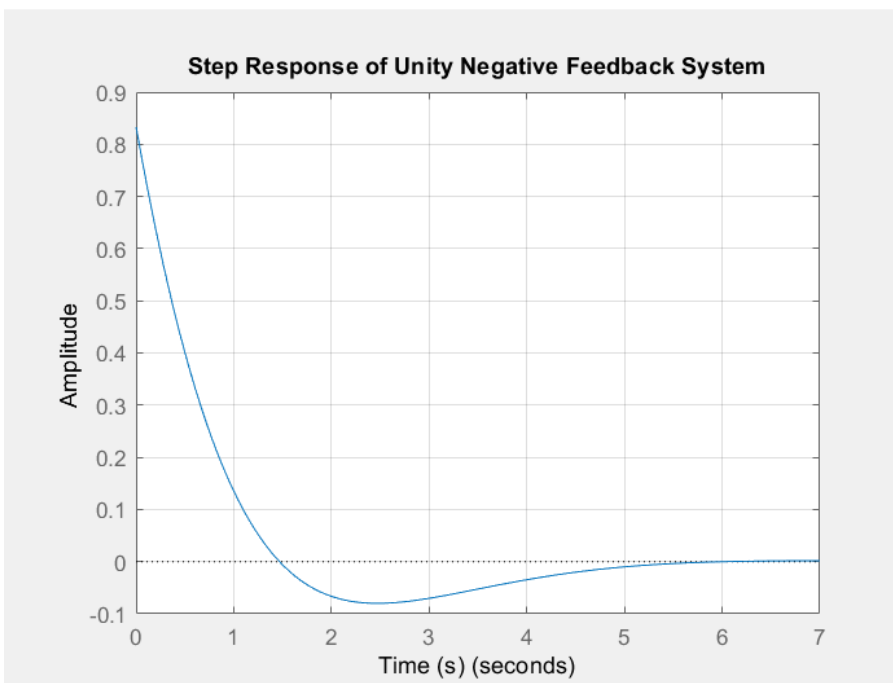

Figure 15: Q4 MATLAB scripts curve.

Question5:

A unity feedback system with a forward transfer function of G(s) is operating with 7% overshoot and settling time of 0.9434 sec

$$G(s)=\frac{K}{(s+2)(s+10)(s+15)}$$

1. Write the MATLAB scripts to plot the root locus
2. Write the MATLAB scripts to determine the desired pole and gain
3. Write the MATLAB scripts to design a PI controller so that the settling time is 105% of the settling time of uncompensated system with zero steady state error
4. Write the MATLAB scripts to plot the step response for both compensated and uncompensated systems
5. Simulate both systems using MATLAB/Simulink to obtain the step response

Part 1:

```
%Yasmin Al Shawawrh
%1220848
%Q5
%part1

s = tf('s');
G = 1 / ((s + 2)*(s + 10)*(s + 15));
T = feedback(G, 1);
figure;
rlocus(T);
title('Root Locus of Open-Loop System');
```

Figure 16: Q5 part 1 code.

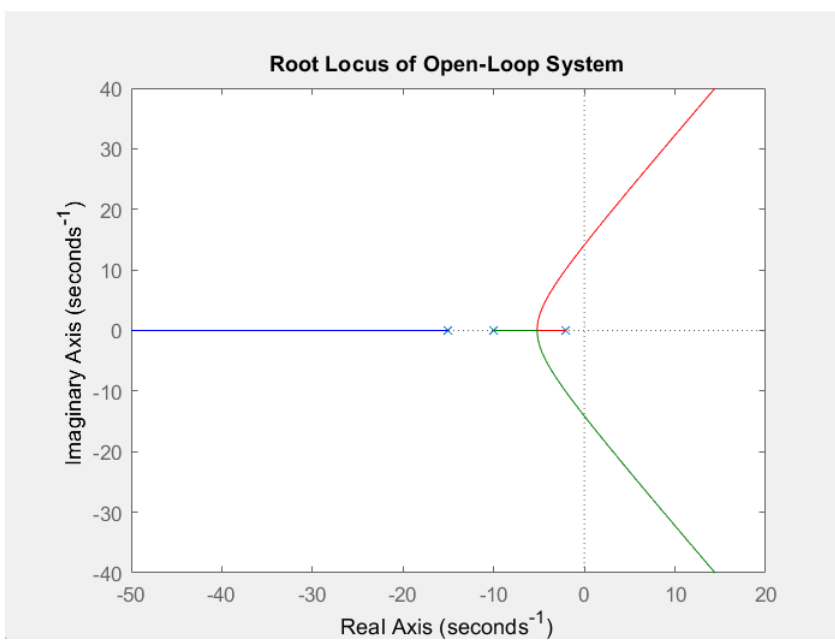

Figure 17: Q5 part 1 curve.

Part 2:

```
%Yasmin Al Shawawrh
%1220848
%Q5
%part2

OS = 7;
Ts = 0.9434;
zeta = -log(OS/100) / sqrt(pi^2 + log(OS/100)^2);
wn = 4 / (zeta * Ts);
sigma = zeta * wn;
wd = wn * sqrt(1 - zeta^2);
s_desired = -sigma + 1i * wd;
den_val = (s_desired + 2) * (s_desired + 10) * (s_desired + 15);
K = -den_val;
fprintf('Desired pole: %.4f %+.4fj\n', real(s_desired), imag(s_desired));
fprintf('Calculated gain K: %.4f\n', real(K));
```

Figure 18: Q5 part 2 code.

```
>> Q5
Desired pole: -4.2400 +5.0090j
Calculated gain K: 497.1206
```

Figure 19: Q5 part 2 the desired pole and gain.

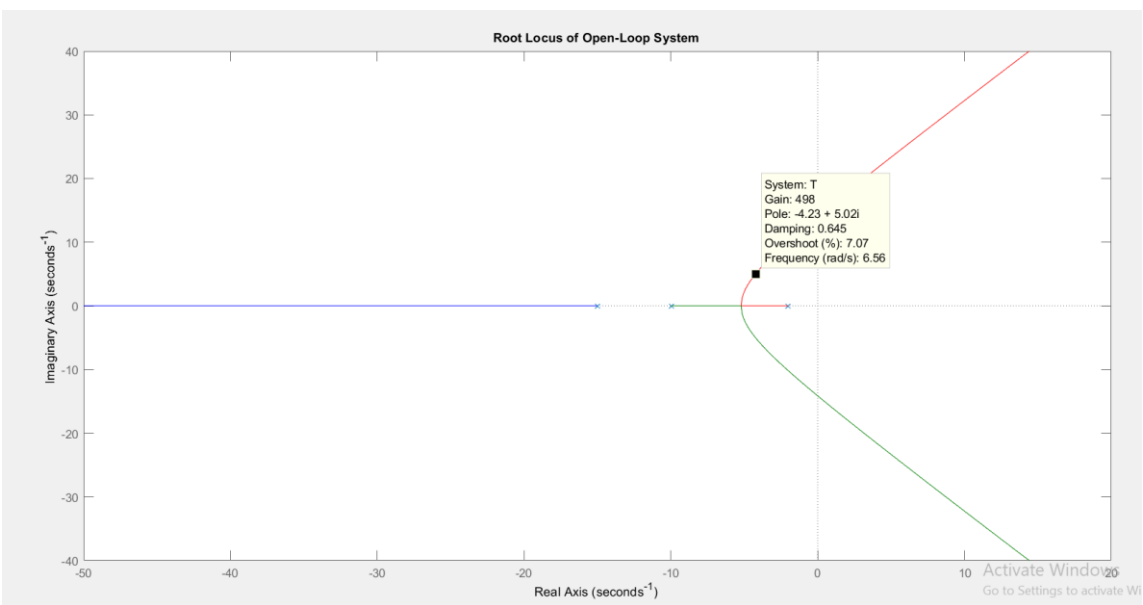

Figure 20: Q5 part 1 curve.

We selected the desired pole on the root locus curve based on system specifications. From that point, the corresponding gain was found to be approximately **498** as shown above.

Part 3:

The new system has zero steady-state error.

The settling time becomes 105% of the original: Ts,new=1.05×0.9434=0.9906 sec

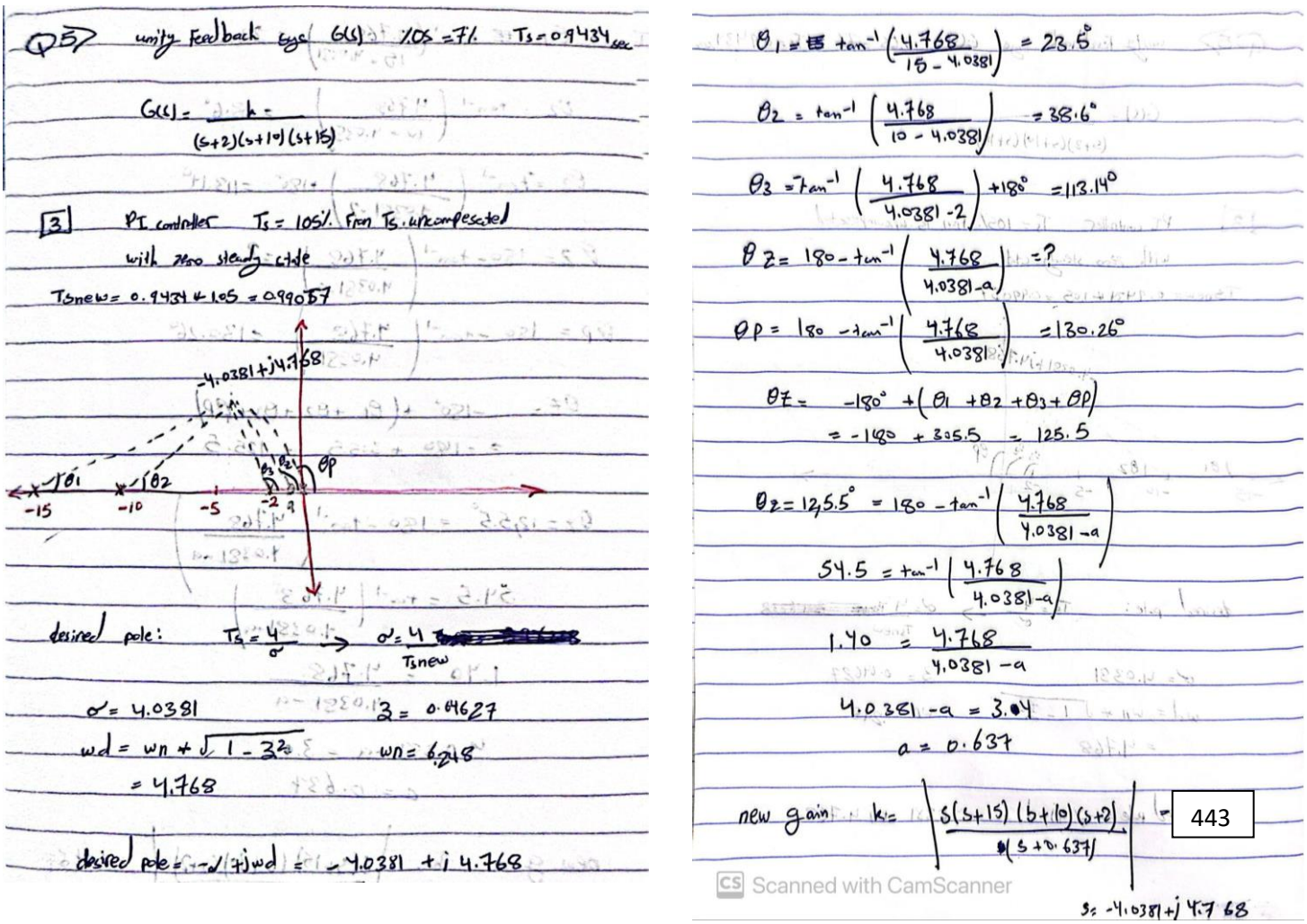

Figure 21: Q5 part 3 by hand solution.

```
%Yasmin Al Shawawrh
%1220848
%Q5
%part3
p1 = -2;
p2 = -10;
p3 = -15;
Ts_new = 1.05 * Ts;
OS = 0.07;
zeta = -log(OS)/sqrt(pi^2 + log(OS)^2);
wn = 4 / (zeta * Ts_new);
sigma = -zeta * wn;
wd = wn * sqrt(1 - zeta^2);
s_des = sigma + 1i*wd;
fprintf('--- Desired Pole ---\n');
fprintf('s_des = %.4f %+.4fj\n\n', real(s_des), imag(s_des));
angle_p1 = angle(s_des - p1) * (180/pi);
angle_p2 = angle(s_des - p2) * (180/pi);
angle_p3 = angle(s_des - p3) * (180/pi);
angle_origin = angle(s_des - 0) * (180/pi);
total_pole_angle = angle_p1 + angle_p2 + angle_p3 + angle_origin;
zero_angle_deg = -180 + total_pole_angle;
zero_angle_rad = deg2rad(zero_angle_deg);
fprintf('\nAngle from poles:\n');
fprintf('Angle p1 (-2): %.2f deg\n', angle_p1);
fprintf('Angle p2 (-10): %.2f deg\n', angle_p2);
fprintf('Angle p3 (-15): %.2f deg\n', angle_p3);
fprintf('Angle from origin (0): %.2f deg\n', angle_origin);
fprintf('Total pole angle: %.2f deg\n', total_pole_angle);
```

```
angle_origin = angle(s_des - 0) * (180/pi);
total_pole_angle = angle_p1 + angle_p2 + angle_p3 + angle_origin;
zero_angle_deg = -180 + total_pole_angle;
zero_angle_rad = deg2rad(zero_angle_deg);
fprintf('\nAngle from poles:\n');
fprintf('Angle p1 (-2): %.2f deg\n', angle_p1);
fprintf('Angle p2 (-10): %.2f deg\n', angle_p2);
fprintf('Angle p3 (-15): %.2f deg\n', angle_p3);
fprintf('Angle from origin (0): %.2f deg\n', angle_origin);
fprintf('Total pole angle: %.2f deg\n', total_pole_angle);
fprintf('Required zero angle: %.2f deg\n', zero_angle_deg);
a = real(s_des) - (imag(s_des)/tan(zero_angle_rad));
fprintf('Zero a = %.4f --> PI Zero at s = -%.4f\n\n', a, a);
PI = tf([1 a], [1 0]);
G = 1 / ((s + 2)*(s + 10)*(s + 15));
num = abs(s_des * (s_des + 2)*(s_des + 10)*(s_des + 15));
den = abs(s_des + a);
K = num / den;
fprintf('--- PI Controller ---\n');
fprintf('Calculated gain K = %.2f\n', K);
fprintf('C(s) = %.2f * (s + %.4f) / s\n\n', K, a);
L = K * PI * G;
T = feedback(L, 1);
T
figure;
step(T);
title('Step Response of PI Compensated System');
xlabel('Time (s)');
ylabel('Amplitude');
```

Figure 22: Q5 part 3 code.

```
s_des = -4.0381 +4.7705j


Angle from poles:
Angle p1 (-2): 113.13 deg
Angle p2 (-10): 38.67 deg
Angle p3 (-15): 23.52 deg
Angle from origin (0): 130.25 deg
Total pole angle: 305.56 deg
Required zero angle: 125.56 deg
Zero a = -0.6273 --> PI Zero at s = --0.6273

--- PI Controller ---
Calculated gain K = 443.56
C(s) = 443.56 * (s + -0.6273) / s


T =

             443.6 s - 278.2
  ----------------------------------------
  s^4 + 27 s^3 + 200 s^2 + 743.6 s - 278.2
```

Figure 23: Q5 part 3 the desired pole and gain.

Part 4:

```
%Yasmin Al Shawawrh
%1220848
%Q5
%part4

s = tf('s');
G = 1 / ((s + 2)*(s + 10)*(s + 15));
K_uncomp = 498; % from part 1
T_uncomp = feedback(K_uncomp * G, 1);
a = 0.6273;
K_comp = 443.56;
PI = tf([1 a], [1 0]);
T_comp = feedback(K_comp * PI * G, 1);
figure;
step(T_uncomp, 'r', T_comp, 'b', 15);
legend('Uncompensated (K = 498)', 'PI Compensated (K = 443.56)');
title('Step Response: Uncompensated vs PI Compensated');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

Figure 24: Q5 part 4 code.

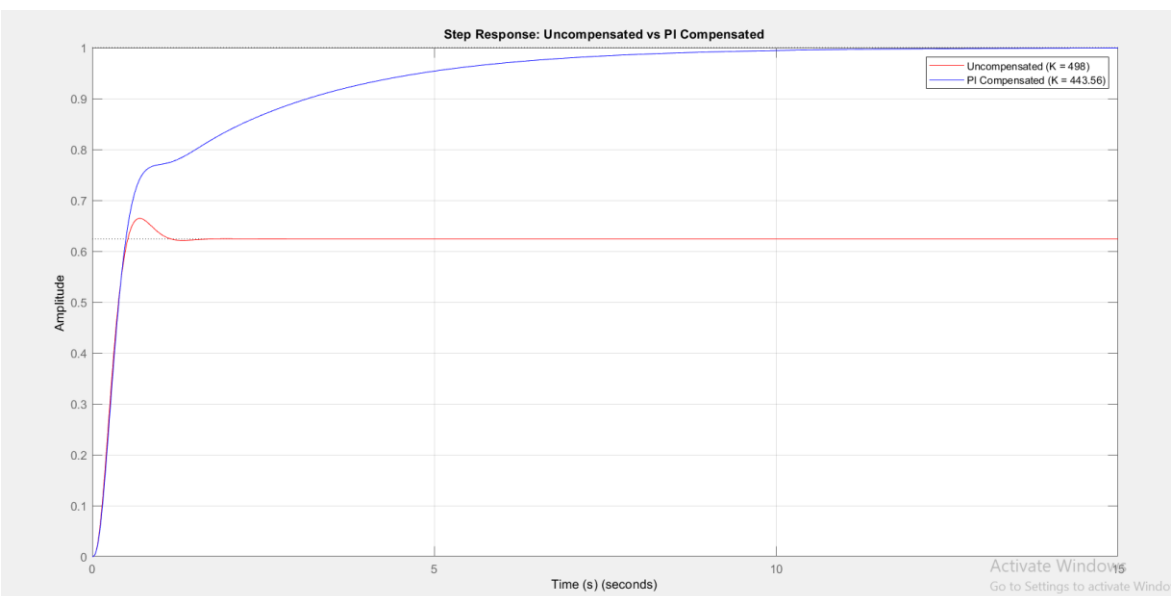

Figure 25: Q5 part 4 curve.

The uncompensated system is faster but doesn't reach the final value. The PI compensated system is slightly slower but achieves zero steady-state error.

Part 5:

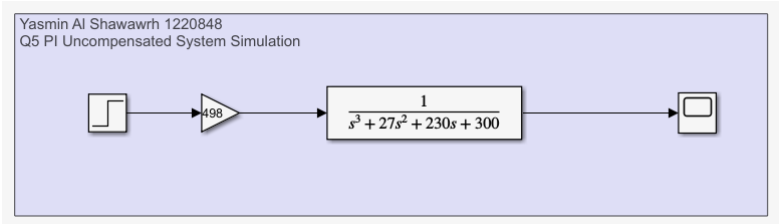

Figure 26: Q5 MATLAB Simulink (uncompensated).

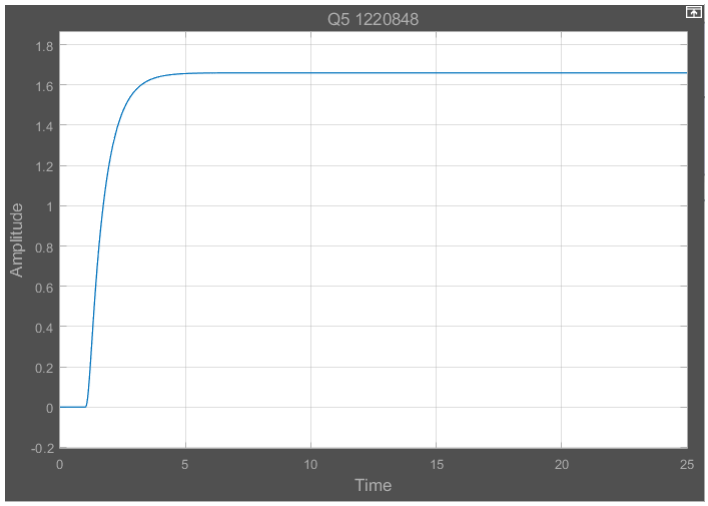

Figure 27: Q5 MATLAB Simulink result (uncompensated).

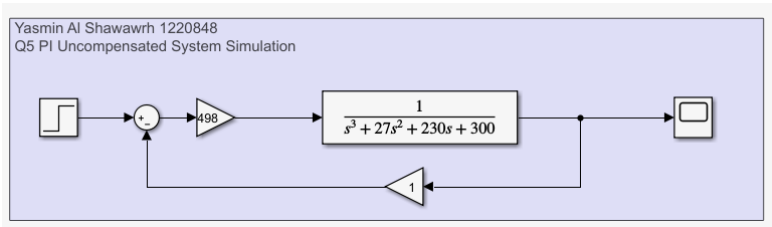

Figure 28: Q5 MATLAB Simulink (uncompensated) with feedback.

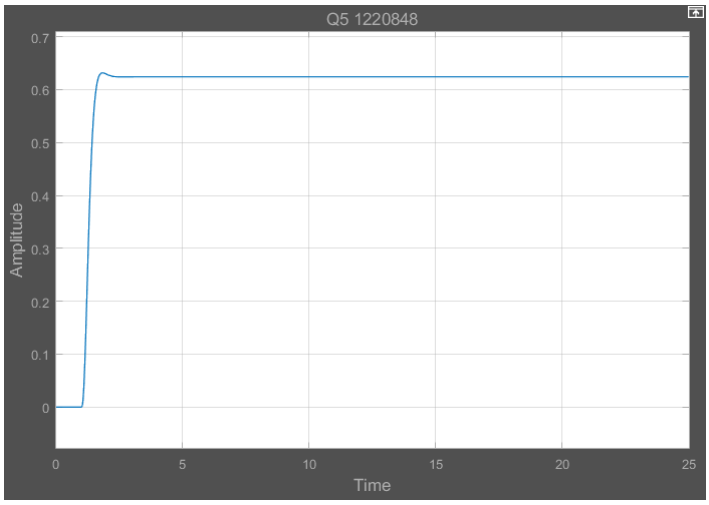

Figure 29: Q5 MATLAB Simulink result (uncompensated) with feedback.

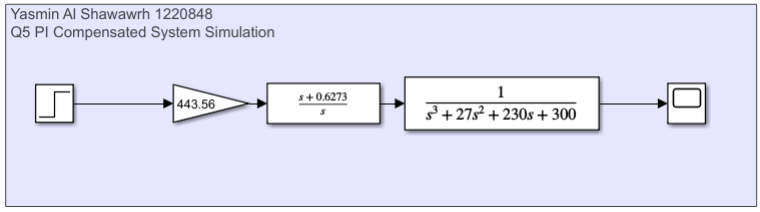

Figure 30: Q5 MATLAB Simulink (compensated).

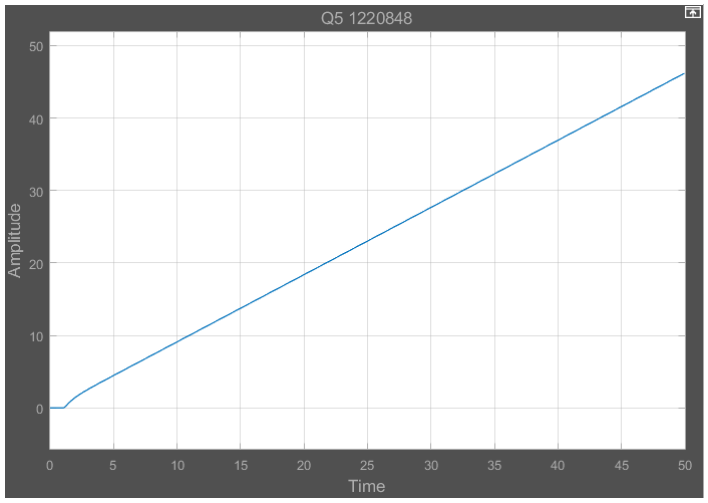

Figure 31: Q5 MATLAB Simulink result (compensated).

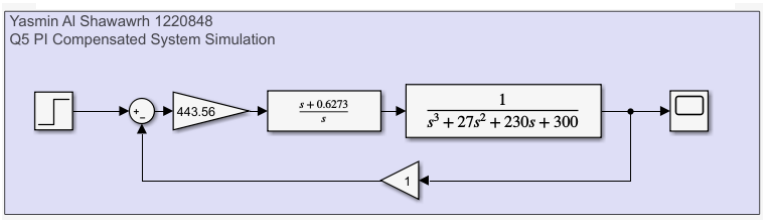

Figure 32: Q5 MATLAB Simulink (compensated) with feedback.

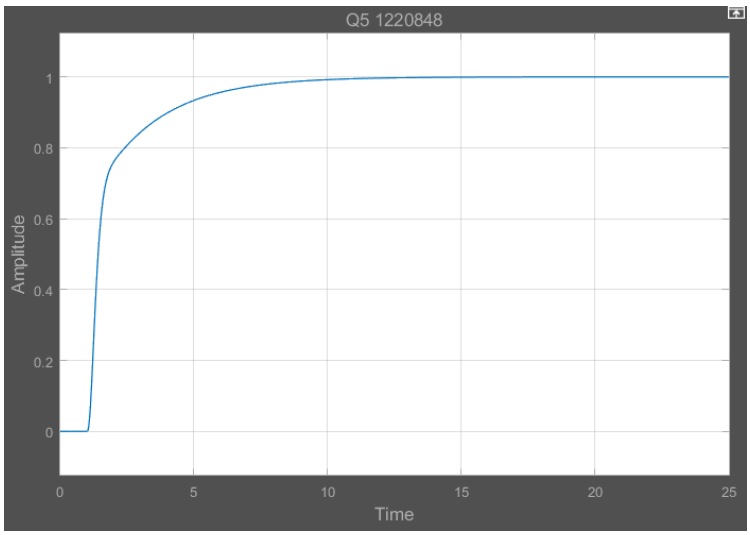

Figure 33: Q5 MATLAB Simulink result (compensated) with feedback.